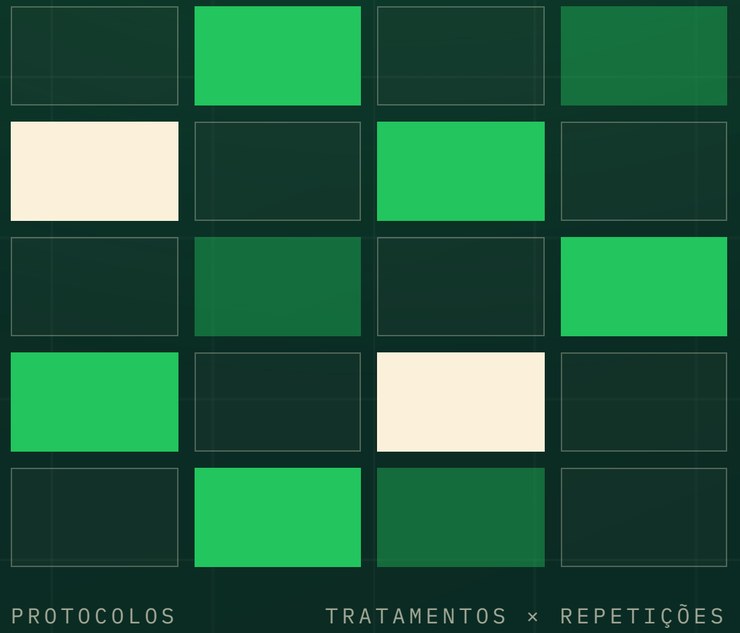
ESTÁGIO SUPERVISIONADO · UFLA 2026

Desenvolvimento de Aplicações Web na Empresa Zeeway

Construção do **front-end** de uma plataforma de protocolos para uma empresa de pesquisa agrícola, substituindo planilhas por registro centralizado e **análise visual automatizada**.



AUTOR
Lucas Henrique Lopes Costa

ORIENTADOR
Prof. Paulo Afonso Parreira Júnior

CURSO
Sistemas de Informação

O CAMINHO DE HOJE

Roteiro da apresentação

01

Introdução

O problema, a empresa e o recorte do trabalho.

02

Fundamentação Teórica

Dados no agronegócio, Scrum e a stack.

03

Metodologia

Como o time trabalhou e como arquitetei o front-end.

04

Atividades Realizadas

O foco desta apresentação: os três módulos que construí.

05

Considerações Finais

Avaliação, limitações e impacto na formação.

01

Introdução

Onde o estágio aconteceu, qual problema a plataforma resolve e o que exatamente foi minha responsabilidade.

CONTEXTUALIZAÇÃO

A pesquisa agrícola gera muito dado. E esse dado vivia em planilhas soltas.

Composição do solo, tratamentos, condições de safra e produtividade, ao longo de anos. Registrar isso em arquivos avulsos cobra um preço.

CONSEQUÊNCIA 01

Rastreabilidade frágil

Sem registro único, ninguém sabe quem alterou o quê.

CONSEQUÊNCIA 02

Regras não aplicadas

A planilha aceita qualquer valor.

CONSEQUÊNCIA 03

Análise manual e lenta

Todo gráfico do relatório era montado à mão.

ONDE O ESTÁGIO ACONTECEU

Zeeway

Empresa lavrense de tecnologia, fundada no ecossistema de inovação da **UFLA**.

15 a 30
colaboradores diretos

≈ 40
clientes diretos

≈ 500
clientes indiretos

O CLIENTE DO PROJETO

Empresa de pesquisa agrícola de Lavras (MG)

Especializada em **análises de solos e sementes**.

O PEDIDO

Uma plataforma própria de protocolos

Registro centralizado, com acesso por perfil.

OBJETIVOS

O que este trabalho se propôs a fazer

OBJETIVO DO ESTÁGIO 01

Aplicar a graduação em ambiente corporativo real

A rotina de desenvolvimento em equipe, fora da sala de aula.

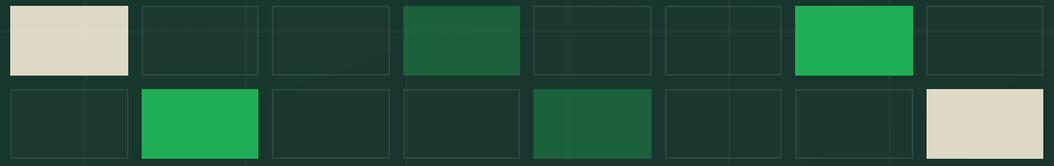


- Equipe estruturada
- Cliente externo e regras não escritas
- Entregas validadas a cada Sprint

OBJETIVO DO RELATÓRIO 02

Documentar de forma estruturada o que foi construído

Registrar as decisões de projeto e a justificativa de cada uma.



- Arquitetura e tecnologias
- Metodologia e coordenação entre equipes
- Desafios técnicos e impacto na formação

DELIMITAÇÃO DO TRABALHO

Minha responsabilidade foi o front-end

O back-end ficou a cargo de outra equipe da Zeeway.

SOB MINHA RESPONSABILIDADE

Camada de interface e experiência do usuário

Telas, formulários, tabelas, estado, consumo da API e geração de gráficos.

OUTRA EQUIPE DA ZEEWAY

API REST, banco de dados e regras no servidor

Persistência, regras no servidor e emissão dos tokens.

02

Fundamentação Teórica

Os conceitos e as tecnologias que sustentaram as decisões tomadas no projeto.

GESTÃO DE DADOS NO AGRONEGÓCIO

Agricultura 4.0: o dado como insumo de decisão

Sensores, IoT, nuvem e análise de dados passam a orientar a decisão no campo.

ANTES

Planilha eletrônica

- Um arquivo por pessoa
- Validação depende das pessoas
- Gráfico manual a cada relatório

DEPOIS

Plataforma web própria

- Registro único, por perfil
- Regras aplicadas pelo sistema
- Gráficos gerados do próprio dado

METODOLOGIAS ÁGEIS

Scrum: iterativo e incremental

A cada **Sprint**, de duração fixa, uma nova versão é entregue.

PAPEL

Product Owner

Prioriza o backlog e define o escopo da Sprint.

PAPEL

Scrum Master

Garante o processo e remove impedimentos.

PAPEL

Time de desenvolvimento

Executa e entrega o incremento.

CERIMÔNIA

Planning

Escopo da Sprint

CERIMÔNIA

Daily

Progresso e impedimentos

EXECUÇÃO

Sprint

Duração fixa

CERIMÔNIA

Review

Incremento ao cliente

CERIMÔNIA

Retrospective

Melhoria do processo

TECNOLOGIAS ADOTADAS

A stack, organizada por responsabilidade

Toda tela do sistema faz quatro coisas: **desenha** o que aparece, **lembra** o que já aconteceu, **busca** os dados no servidor e precisa ser **montada**. Cada coluna resolve uma delas.

APRESENTAÇÃO · DESENHA

React + TypeScript

Componentização e tipagem. Material UI, TanStack Table e ApexCharts.

ESTADO · LEMBRA

Zustand + Immer

Estado global e sessão persistida. Formik e Yup nos formulários.

DADOS · BUSCA

Axios + React Query

Token em toda chamada. Cache que evita requisição repetida.

BUILD · MONTA

Vite

Build e servidor de desenvolvimento com recarga instantânea.

BIBLIOTECAS DE APOIO

`jwt-decode · token` `mathjs · fórmulas` `JSZip · fotos em .zip` `use-react-screenshot · PNG` `qrcode · campo`

03

Metodologia e Processo

Como o time trabalhou, como os requisitos foram levantados e como o front-end foi arquitetado.

SCRUM APLICADO AO PROJETO

Sprints de duas semanas

- **Planning:** o backlog priorizado vira Sprint Backlog
- Itens acompanhados no **Azure DevOps Boards**
- **Dailies:** progresso e impedimentos
- **Review** com o cliente, **retrospective** com o time

QUADRO DE TAREFAS

A FAZER	EM ANDAMENTO	EM REVISÃO	CONCLUÍDO
#412	#398	#377	#351
#415	#402		#358
#419			#366

Entregas incrementais e validação contínua, com menos retrabalho.

LEVANTAMENTO DE REQUISITOS

As regras estavam na prática, não no papel

Boa parte das regras era específica do domínio e vivia na rotina da equipe, não em documento.

Tipo do protocolo
RET OU COMERCIAL

×

Cultura selecionada
DEFINIDA NO PROTOCOLO

×

Classe do experimento
UMA OU MAIS CLASSES

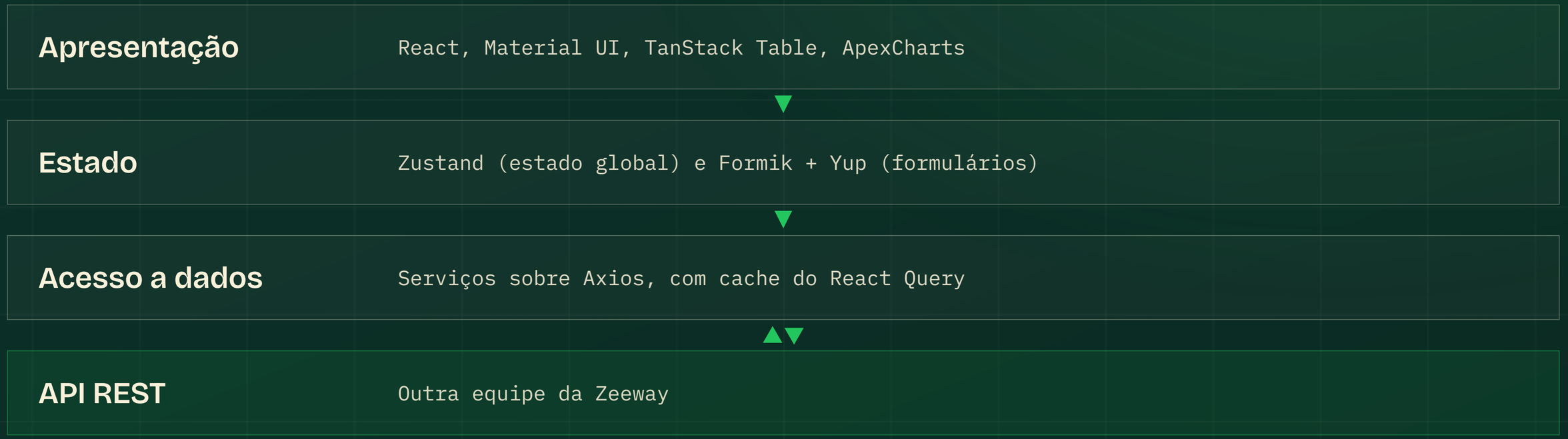
▼

Obrigatoriedade e formato de cada campo do formulário

- Nada disso estava registrado
- Cada variação era esclarecida com o cliente
- A regra acordada virava esquema de validação

ARQUITETURA DO FRONT-END

Três camadas, três responsabilidades



Cada camada tem um motivo único para mudar.

COORDENAÇÃO ENTRE AS EQUIPES

Como front e back andaram em paralelo

Paralelizar o trabalho a partir de uma interface comum, acordada antes do código.

ETAPA 01	ETAPA 02	ETAPA 03	ETAPA 04
Contrato acordado	**Trabalho simultâneo**	**Integração**	**Ajuste**
Endpoint, método e formato dos dados	Back faz o endpoint. Front constrói a tela com mock	Troca o mock pela API real	Ajustes na daily ou na planning

Nenhuma das equipes ficava permanentemente em ritmo diferente da outra.

04

Atividades Realizadas

A plataforma de protocolos, módulo a módulo: o que foi construído, as decisões técnicas por trás e os desafios enfrentados em cada etapa.

VISÃO GERAL

Três módulos, selecionados por relevância e complexidade técnica

MÓDULO 01

Autenticação e Controle de Acesso

Sessão com JWT, renovação transparente e acesso por perfil.

MÓDULO 02

Registros Agrícolas

O núcleo. Listagem, cadastro e edição dos protocolos.

MÓDULO 03

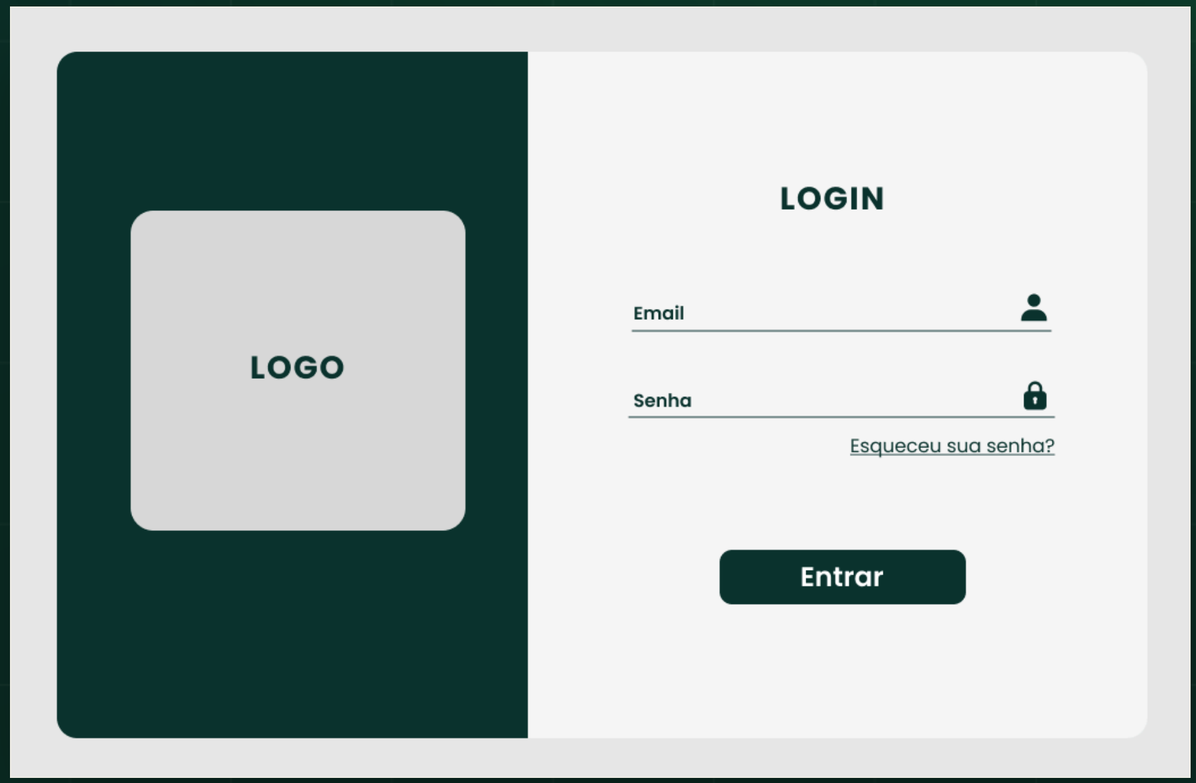
Análise e Dashboards

O diferencial. Gráficos automáticos e exportação com a marca do cliente.

MÓDULO 01 · AUTENTICAÇÃO

O ponto de entrada de todos os usuários

- **Identidade visual** do cliente
- E-mail, senha e recuperação
- A API devolve um par de **tokens JWT**
- Sessão persistida entre acessos



TELA DE LOGIN DA PLATAFORMA

Renovação de sessão transparente para o usuário

- O token é anexado uma vez, para todas as chamadas
- O interceptador só age no **401**
- Renova o token e **reenvia** a requisição
- Sem logout no meio de um formulário longo

```
axiosInstance.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;
    const status = error.response?.status;
    if (status === 401 && !originalRequest._retry) {
      const { refreshToken } =
        loginStorePersist.getState().session;
      originalRequest._retry = true;
      const response = await axiosInstance.post(
        "/user/refreshToken", { refreshToken }
      );
      const newToken = response.data.data.accessToken;
      loginStorePersist.getState()
        .setLoginInfo({ data: response.data });
      originalRequest.headers["Authorization"] =
        `Bearer ${newToken}`;
      return axiosInstance(originalRequest);
    }
    return Promise.reject(error);
  }
);
```

MÓDULO 01 · AUTENTICAÇÃO

Controle de acesso por perfil, no nível da rota

PERFIL

Administrador

Acesso completo.

PERFIL

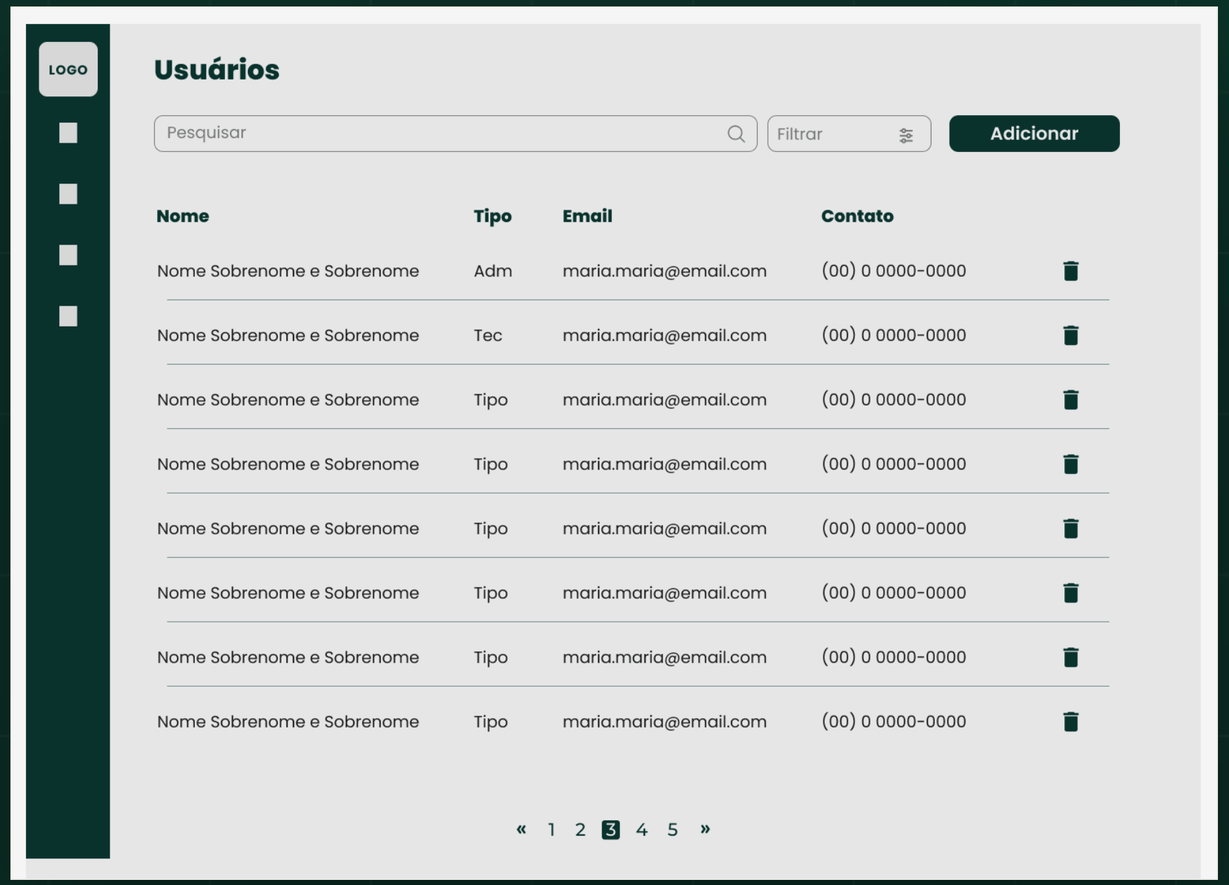
Técnico

Registro e consulta de protocolos.

- Um guardião decide antes de renderizar
- Sem permissão, redireciona

```
const ProtectedRouteGuard = ({ children, adminRoute }) => {
  const { isAuthenticated, userData: { roleId } } =
    loginStorePersist((state) => state);
  const { isUserRole } = useRoles();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }
  if (adminRoute && !isUserRole("administrador", roleId)) {
    return <Navigate to="/" replace />;
  }
  return children;
};
```



GESTÃO DE USUÁRIOS, ACESSÍVEL APENAS PELO ADMINISTRADOR

MÓDULO 01 · AUTENTICAÇÃO

Gestão de usuários

- Visualizar, adicionar, editar e excluir
- **Busca textual** e filtro por perfil
- **Modais**, para não perder o contexto
- **Formik** no estado, **Yup** na validação

MÓDULO 02 · REGISTROS AGRÍCOLAS

Listagem de protocolos com TanStack React Table

CÓDIGO	TIPO	CLIENTE	CLASSE	CULTURA	STATUS
ADAMT_V1_SAFRA23/24	Ret	Cliente 01	Fertilizantes	Milho	Concluído
BXQRT_V2_SAFRA23/24	Comercial	Cliente 02	Nutrição	Soja	Em andamento
CLMPT_V1_SAFRA24/25	Ret	Cliente 03	Milharal	Café	Não iniciado

RECURSO

Ordenação e paginação

Sobre um volume crescente de protocolos.

RECURSO

Filtragem

Código, tipo, cliente, classe, cultura, status.

RECURSO

QR Code

Escaneado **em campo**, abre o protocolo.

RECURSO

Exportação

Saída para planilha.

MÓDULO 02 · REGISTROS AGRÍCOLAS

O maior desafio técnico do projeto

Dezenas de campos em um único protocolo, organizados em **oito abas**.

Informações | Avaliações | Tratamentos | Local | Solo | Aplicações | Manejo | Dashboard

CAMPOS OBRIGATÓRIOS

O mínimo para um protocolo existir

Tipo, código, cliente, classes, status, tratamentos e repetições.

CAMPOS OPCIONAIS

Preenchidos conforme a pesquisa avança

Cultura, variedade, datas, adubação, espaçamento e delineamento.



ABA DE AVALIAÇÕES, COM TABELAS POR MOMENTO E FOTOGRAFIAS DE CAMPO

MÓDULO 02 · REGISTROS AGRÍCOLAS

Uma estrutura que muda de forma

- As **colunas** variam conforme o número de repetições
- Linhas adicionadas conforme a necessidade
- Campos-**fórmula** calculados em execução com **mathjs**
- **Fotografias de campo** por momento de avaliação

Todas as fotos de uma avaliação em um único arquivo

- Baixar dezenas de fotos uma a uma inviabilizava o trabalho
- As imagens são buscadas na API e viram blobs
- Baixadas **em paralelo** com Promise.all
- O **JSZip** agrupa e entrega um .zip

```
const handleDownloadImages = async (evaluation) => {
  getImages(evaluation.id, {
    onSuccess: async ({ data }) => {
      const images = data.data;
      const blobs = await Promise.all(
        images.map((img) =>
          fetch(img.url).then((res) => res.blob())
        )
      );
      const zip = new JSZip();
      blobs.forEach((blob, i) =>
        zip.file(`${images[i].title}.jpg`, blob)
      );
      const zipFile = await zip.generateAsync({
        type: "blob",
      });
      saveFile(zipFile, evaluation.name);
    },
  });
};
```

Traduzir a regra do agrônomo em esquema de validação

- Traduzir a regra do cliente foi o mais difícil
- **required** com **nullable**: nulo enquanto preenche, exigido no envio
- `array().min(1)` garante ao menos uma classe selecionada
- A mensagem da regra é o que o usuário lê

```
const validationSchema = Yup.object().shape({
  isRET: Yup.bool().required("O campo e obrigatorio"),
  code: Yup.string().required("O campo e obrigatorio"),
  culture: Yup.string(),
  experimentalDesign: Yup.string().nullable(),
  numberOfTreatments: Yup.number()
    .required("O campo e obrigatorio").nullable(),
  numberOfRepeats: Yup.number()
    .required("O campo e obrigatorio").nullable(),
  plantingDate: Yup.string().nullable(),
  harvestDate: Yup.string().nullable(),
  classes: Yup.array().min(1, "O campo e obrigatorio"),
  clientId: Yup.string().required("O campo e obrigatorio"),
});
```

MÓDULO 03 · ANÁLISE E DASHBOARDS

Aqui está o que a planilha não fazia: transformar dado bruto em gráfico, sem ninguém montar nada.

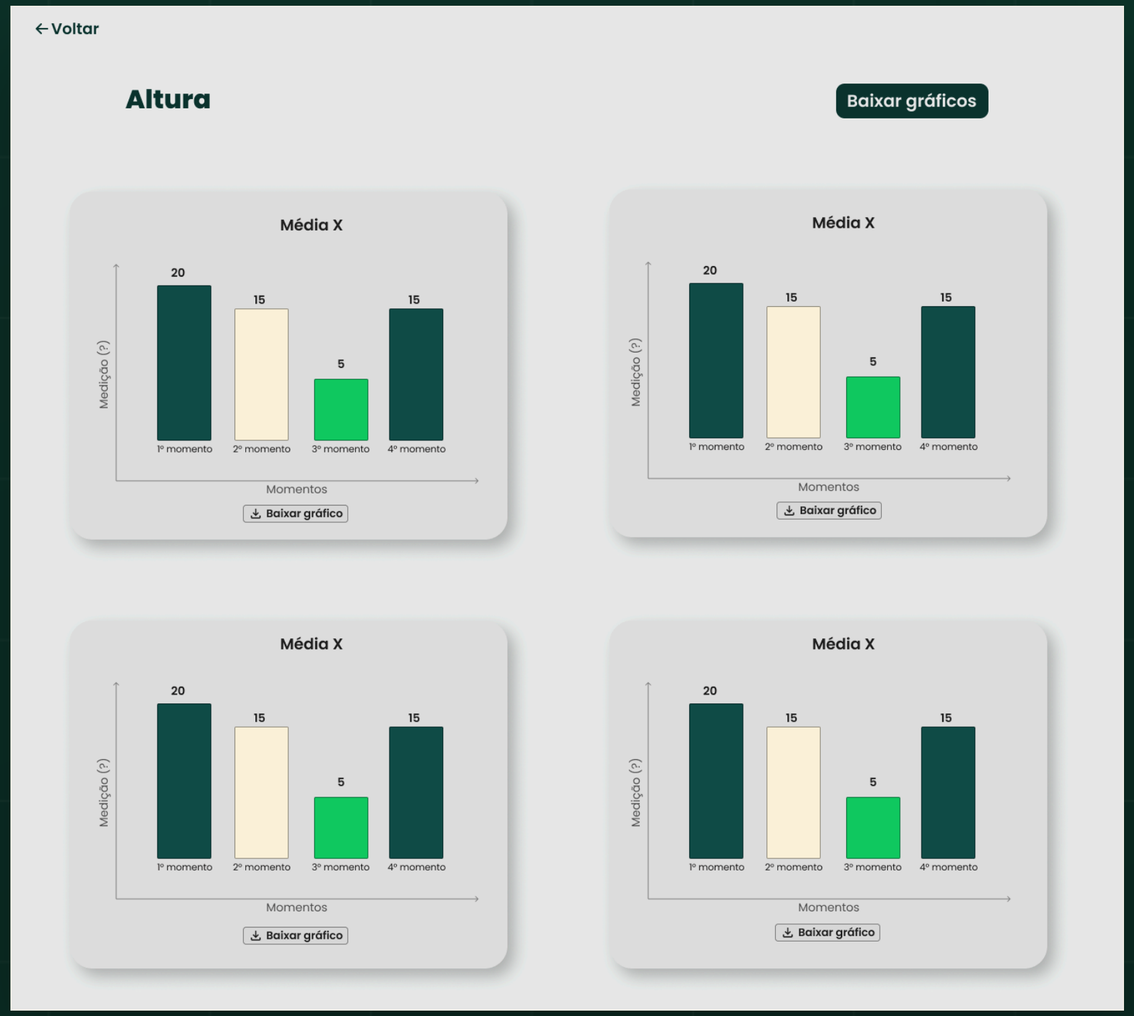
MÓDULO 03 · ANÁLISE E DASHBOARDS

Do dado de avaliação à imagem exportada

ETAPA 01	ETAPA 02	ETAPA 03	ETAPA 04	ETAPA 05
Dados do back-end	**Mapeamento para séries**	**Opções do ApexCharts**	**Renderização**	**Exportação PNG**
Já consolidados pelo servidor	Rótulos e valores	Tipo, cores e rótulos	Um gráfico por card	Com a identidade visual

```
const series = [{ name: xAxisTitle, data: yAxis }];

const options: ApexOptions = {
  chart: { type: "bar", toolbar: { show: false } },
  plotOptions: { bar: { distributed: true } },
  dataLabels: { enabled: true },
  xaxis: { categories: xAxis },
  yaxis: { max: maxValue + 1 },
};
```



GRÁFICOS GERADOS AUTOMATICAMENTE PARA UMA AVALIAÇÃO

MÓDULO 03 · ANÁLISE E DASHBOARDS

Nenhum ajuste manual

- O componente monta **sozinho** séries e opções
- Barras comparam tratamentos e momentos de avaliação
- Um **card por gráfico**, com download individual
- Valores sobre as colunas, para comparar de imediato

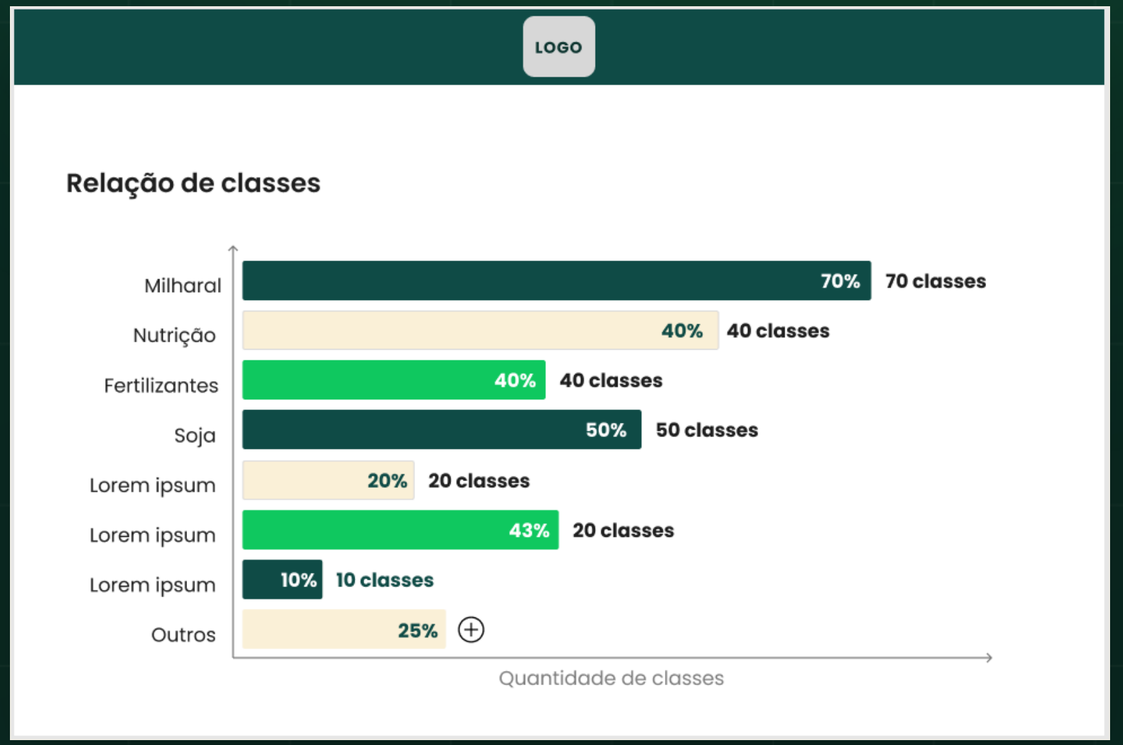
MÓDULO 03 · ANÁLISE E DASHBOARDS

Pronto para o relatório do cliente

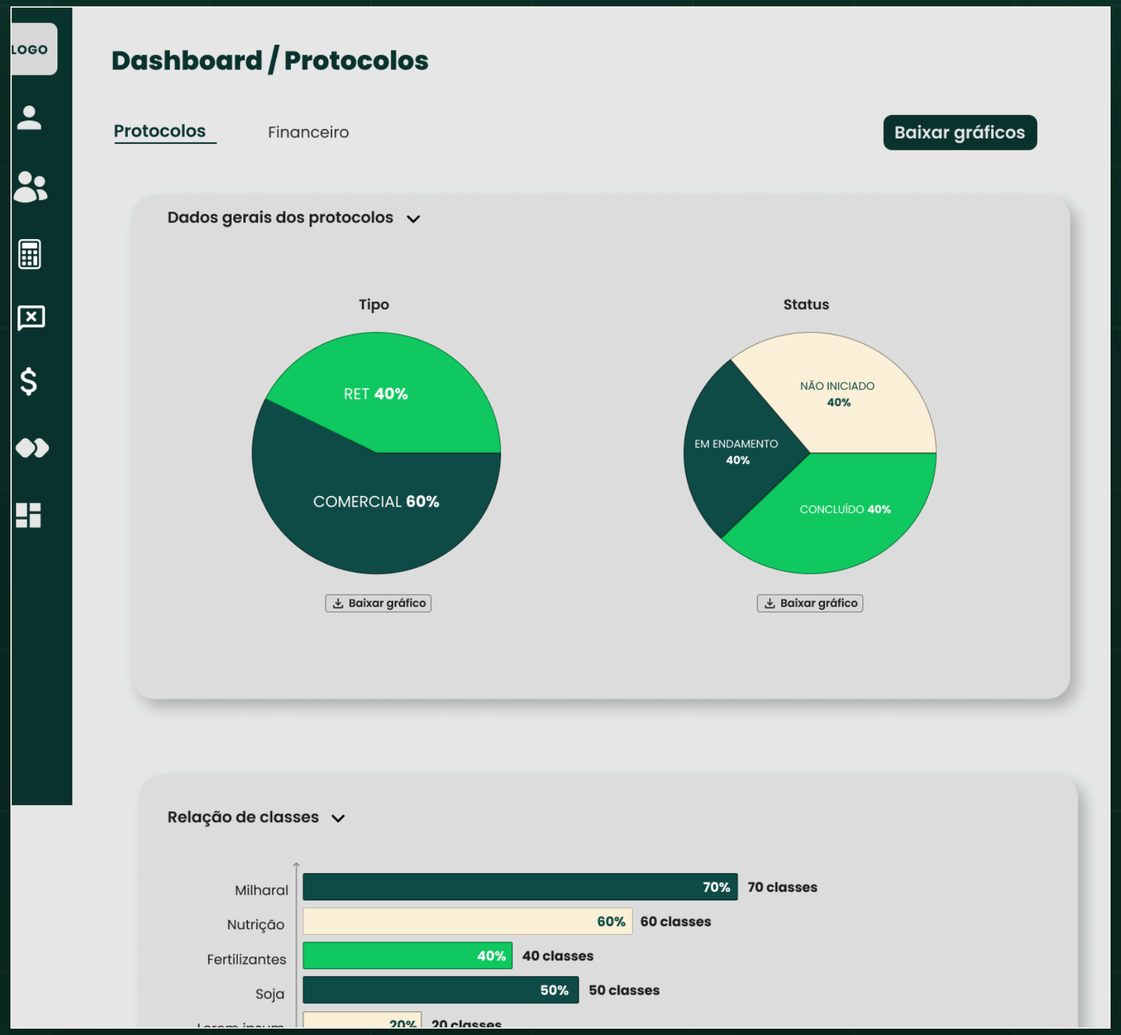
- Um elemento oculto reúne gráfico, **logotipo** e título
- **use-react-screenshot** devolve a imagem em base64
- Um link criado na hora força o download

```
const [_, takeScreenshot] = useScreenshot();

const downloadChart = async (format) => {
  const imageBase64 =
    await takeScreenshot(imageRef.current);
  const link = document.createElement("a");
  link.href = imageBase64;
  link.download =
    `${imageTitle} - ${chartTitle}.${format}`;
  link.click();
};
```



PNG EXPORTADO, COM LOGOTIPO E TÍTULO DOS DADOS



DASHBOARD GERAL, COM A VISÃO CONSOLIDADA DOS PROTOCOLOS

MÓDULO 03 · ANÁLISE E DASHBOARDS

A visão de todos os protocolos

- Protocolos por **classe**, **cultura**, **cliente** e **status**
- Dados consolidados pelo back-end, buscados com **React Query**
- O cache mantém tudo atualizado sem recarregar
- Mesmos vetores de rótulos e valores dos gráficos individuais

MÓDULO 03 · O DESAFIO TÉCNICO

Muitos gráficos na mesma tela custam caro

SINTOMA

Lentidão perceptível

Renderizar vários gráficos ao mesmo tempo travava a interface.

CAUSA

Tudo renderizado de uma vez

Gráficos fora da tela eram montados junto com os visíveis.

SOLUÇÃO

Carregamento sob demanda

Com **lazy loading**, o gráfico só é montado quando aparece.

05

Considerações Finais

Avaliação do que foi entregue, limitações honestas, trabalhos futuros e o que o estágio deixou na formação.

AVALIAÇÃO DO PRODUTO

O objetivo central foi atendido no plano técnico

Sair da planilha para uma plataforma própria, com acesso controlado e análise automatizada.

ENTREGA 01

Centralização

Cadastro completo dos protocolos, com validação declarativa.

ENTREGA 02

Controle de acesso

Cada colaborador vê apenas o que é do seu papel.

ENTREGA 03

Análise automatizada

Gráficos gerados do próprio dado, exportáveis com a marca do cliente.

O QUE É PRECISO DIZER COM HONESTIDADE

Dificuldades, limitação e trabalhos futuros

DIFICULDADES SUPERADAS

Regras de negócio e performance

- Traduzir o domínio agronômico para a lógica dos formulários
- Desempenho na renderização de múltiplos gráficos

LIMITAÇÃO OBSERVADA

Adoção inicial reduzida

- A adoção inicial foi baixa
- Atribuível à **mudança de cultura**, não a falha técnica

TRABALHOS FUTUROS

O que viria em seguida

- Acompanhar a adoção no tempo
- Testes de usabilidade com os colaboradores
- Novos tipos de análise nos dashboards

IMPACTO DO ESTÁGIO NA FORMAÇÃO

A primeira experiência com software de verdade

Produto real, equipe estruturada e cliente externo.

PLANO TÉCNICO

Competências consolidadas

React e TypeScript, estado global, formulários, integração com API e, sobretudo, **visualização de dados**.

PLANO INTERPESSOAL

Comunicação e trabalho em equipe

Traduzir necessidade de negócio em solução técnica exigiu diálogo constante.

DIREÇÃO DE CARREIRA

Interfaces orientadas a dados

Visualização de dados passou a orientar meu interesse profissional.

O CAMINHO INVERSO

O que a graduação deu, e o que faltou

DISCIPLINAS QUE SUSTENTARAM O TRABALHO

- **Engenharia de Software:** requisitos e métodos ágeis
- **POO:** modularidade virou componentização
- **Banco de Dados:** as estruturas da API
- **IHC:** usabilidade e organização das telas
- **AED:** vetores e estruturas aninhadas

LACUNAS PERCEBIDAS

- **Frameworks modernos de front-end**, vistos de forma superficial
- **Testes automatizados de interface**, pouco trabalhados no curso
- **Integração contínua e versionamento em equipe**, com pull requests
- No mesmo grupo, **conteinerização e infraestrutura**

Cobrir isso teria reduzido a curva dos primeiros dias.

OBRIGADO PELA ATENÇÃO

Perguntas?

AUTOR

Lucas Henrique Lopes Costa

ORIENTADOR

Prof. Paulo Afonso Parreira Júnior

INSTITUIÇÃO

UFLA · Sistemas de Informação · 2026



EQUIPE ZEEWAY